

Project:

(2)

File Packer Unpacker

Date: 8 / 8 / 2026.

① This project is used to perform process monitoring activity.

② By using this project we can fetch data from all files and merge it into one file. As well as we can also extract all packed file whenever required.

③ Platform Required :

Windows NT platform or Linux.

④ Architectural requirement :

Intel 32 bit processor.

⑤ User Interface :

Graphical User Interface.

⑥ Technology used :

Java Programming

⑦ Features provided by File Packer - Unpacker :

This project is divided into two parts as packing and unpacking.

⑧ Packing Activity :

- In case of Packing Activity we accept directory name and file name from user.

- We have to create new regular files as the name specified by user.

Date: / /

- Now open the directory and traverse each file from that directory. In newly created file write metadata as header and actual file data in Sequence.
- while writing data perform encryption.
- Each name of file, its size and checksum should be written in log file which gets created in system directory.
- After packing display packing report.

③ Unpacking Activity :-

- In case of Unpacking Activity we accept packed file name from user.
- for authentication of packed file use any logic like magic Number.
- open the packed file in read mode and perform below activity as
 - Read header
 - from the name specified in header create new file.
 - Write data into newly created file from packed file.
 - Repeat above all steps still we reached at the end of the file unpacked file.

Date: / /

- After unpacking display unpacking report.
- Main purpose of this project is to merge large amount of files into one file by avoiding memory wastage.
- We also provided data security using the concept of Encryption.
- For next level data security we add MD5 checksum check and primary header.

⊕ Diagram.

File :-

Packing

Input Folder

Data

A.txt

AAAAA
AAAAA

B.txt

BBBBB
BBBBB
BBBBB

C.txt

CCCCC
CCCCC
CCCCC

MarvellousPac.txt

A.txt 10

A A A A
A A A A

B.txt 20

BBBBBBBBBB
BBBBBBBBBB

C.txt 30

CCCCCCCCC
CCCCCCCCC
CCCCCCCCC

→Header

→Data

→Header

→Data

→Header

→Data

Output Files

A.txt

AAAAA
AAAAA

B.txt

BBBBB
BBBBB
BBBBB

C.txt

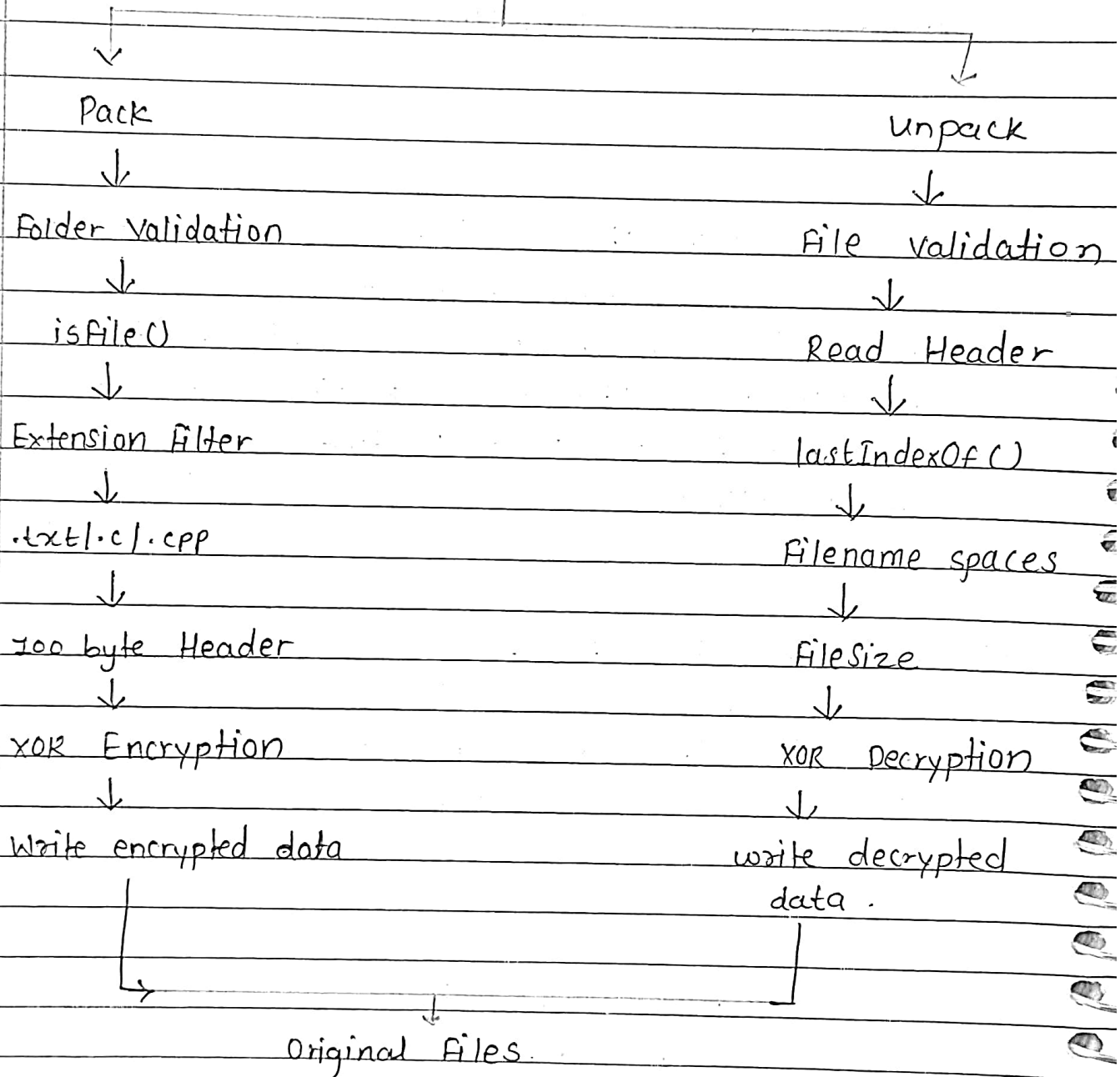
CCCCC
CCCCC
CCCCC

unpacking

Date: / /

⊕ features of project : floco

FILE PACKER - UNPACKER



* Frontend : GUI

- ① 1) JFrame → Represents the main Application Window.
- 2) JPanel → Represents a section/containers inside the window.
- 3) JLabel → Represents Text → display info File
- 4) JTextField → Input Box ^{accept user input}
- 5) JButton → Button
- 6) JFileChooser → File/folder selection dialog.
- 7) JPasswordField → Represents password.

- frame.setSize (width, height) pixels

frame.setVisible (true)

frame.setDefaultCloseOperation (JFrame.EXIT_ON_CLOSE);

- JPanel packerPanel = new JPanel ();

- frame.setLayout (new GridLayout (^{row}1, ^{col}2)); = 1 row x 2 col.

packerPanel.setBorder (BorderFactory.createTitleBorder ("PACKER"));

- JLabel folderLabel = new JLabel ("Folder :");

JTextField folderText = new JTextField ();

- packerLabel.setBounds (40, 50, 100, 30);

^{left margin}
^{vertical position} ^{width} ^{height}

① ActionListener

user clicks Browse



Action listener triggered



JFileChooser opens



User selects folder



getSelectedFile()



getAbsolutePath()



folderText.setText(path)

GUI Design:-

FILE PACKER - UNPACKER	
File Management utility	
PACKER Folder : Browse Pack File : Browse Pack	UNPACKER Packed File : Browse Unpack

① cook-flocw (Packing)

Open Application



click folder Browse



select Source folder



select Pack File



click PACK

Backend
Logic



Files are read ... (iret = fobj.read (Buffer) != -1)



Headers are created ... (f.getName(), f.length)



Encryption ... Buffer[k] = (byte) (Buffer[k] ^ key);

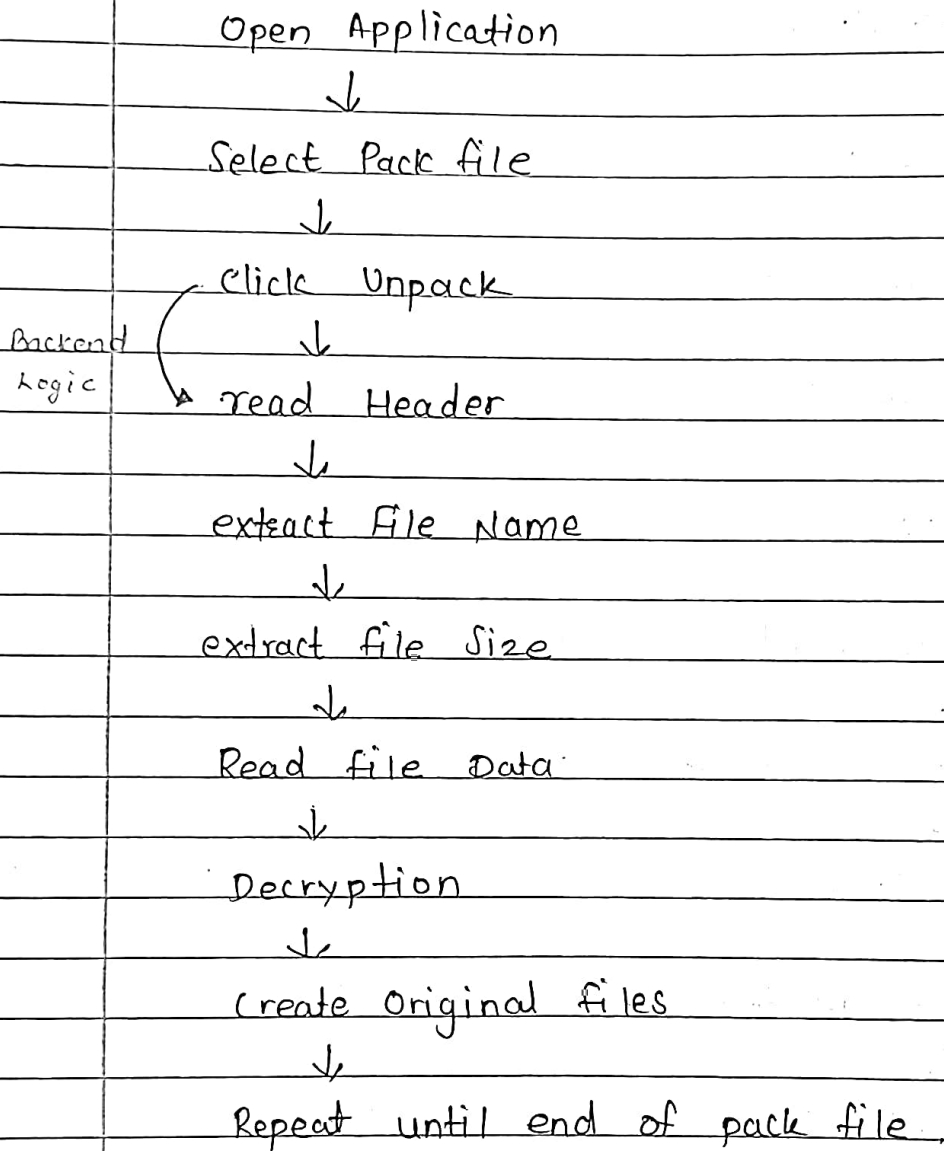


Data is written ... fobj.write (Buffer, 0, iret)



Pack File Created ... (-createNewFile ())

② Workflow (Unpacking)



C Code

```
1  //////////////////////////////////////
2  // Final Packing Code
3  //////////////////////////////////////
4
5  import java.io.*;
6
7  class FilePacker
8  {
9      private static final int HEADER_SIZE = 100;
10     private static final int BUFFER_SIZE = 1024;
11     private static final int KEY = 4242;
12
13     public static boolean pack(String FolderName, String PackFileName) throws
Exception
14     {
15         int iRet = 0;
16         int Size = 0;
17         int i = 0, j = 0;
18
19         String header = ""; //IMP
20
21         FileOutputStream foobj = null;
22         FileInputStream fiobj = null;
23
24         byte Buffer[] = new byte[BUFFER_SIZE];
25         byte bHeader[] = null;
26
27         File fobjfolder = new File(FolderName);
28
29         if((fobjfolder.exists()) && (fobjfolder.isDirectory()))
30         {
31             System.out.println("Folder exists");
32
33             File fobjpack = new File(PackFileName);
34             fobjpack.createNewFile(); // Pack file gets created
35
36             foobj = new FileOutputStream(fobjpack);
37
38             File fArr[] = fobjfolder.listFiles();
39
40             System.out.println("Number of files in folder : "+fArr.length);
41
42
43             for(i = 0; i < fArr.length; i++)
44             {
45
46                 if(fArr[i].isFile()) //$
47                 {
48                     String FileName = fArr[i].getName().toLowerCase(); //$
49
```

```

50         if(FileName.endsWith(".txt") || //File extension filter
51            FileName.endsWith(".c") ||
52            FileName.endsWith(".cpp") ||
53            FileName.endsWith(".java")
54        )
55        {
56            fiobj = new FileInputStream(fArr[i]);
57
58            header = header + fArr[i].getName();
59            header = header + " ";
60            header = header + fArr[i].length();
61
62            Size = 100 - header.length();
63
64            for(j = 1; j <= Size; j++)
65            {
66                header = header + " ";
67            }
68
69            bHeader = header.getBytes();
70
71            // Write file name and size
72
73            foobj.write(bHeader);
74
75            // Read, encrypt and write file data // $
76            while((iRet = fiobj.read(Buffer)) != -1)
77            {
78                for(int k = 0; k < iRet; k++)
79                {
80                    Buffer[k] = (byte)(Buffer[k] ^ KEY); //XOR Operation
81                }
82
83                foobj.write(Buffer, 0, iRet);
84            }
85            fiobj.close();
86            header = ""; //IMP
87
88        }
89    }
90
91    foobj.close();
92    return true;
93 }
94 else
95 {
96     return false;
97 }
98 }
99 }
100 }

```

C Code

```
1  //////////////////////////////////////////////////
2  // Final UnPacking Code
3  //////////////////////////////////////////////////
4
5  import java.io.*;
6
7  class FileUnpacker
8  {
9      private static final int HEADER_SIZE = 100;
10     private static final int KEY = 4242;
11
12     public static boolean unpack(String PackFileName) throws Exception
13     {
14         File fpackobj = null;
15         FileInputStream fiobj = null;
16         FileOutputStream foobj = null;
17         byte Header[] = new byte[HEADER_SIZE];
18         String strHeader = null;
19         File NewFile = null;
20         byte Buffer[] = null;
21         int iRet = 0;
22
23         fpackobj = new File(PackFileName);
24
25         if(fpackobj.exists())
26         {
27             fiobj = new FileInputStream(fpackobj);
28
29             // Read header
30             while((iRet = fiobj.read(Header, 0, HEADER_SIZE)) != -1)
31             {
32                 strHeader = new String(Header);
33
34                 strHeader = strHeader.trim();
35
36                 int iPos = strHeader.lastIndexOf(' '); //$ handles whitespaces inside
filenames
37
38                 String FileName = strHeader.substring(0, iPos); //$
39                 String FileSize = strHeader.substring(iPos + 1); //$
40
41                 System.out.println("File name : " + FileName);
42                 System.out.println("File size : " + FileSize);
43
44                 NewFile = new File(FileName);
45                 NewFile.createNewFile();
46
47                 foobj = new FileOutputStream(NewFile);
48
49                 int FileSizeInt = Integer.parseInt(FileSize);
```

```
50
51     Buffer = new byte[FileSizeInt]; //$
52
53     // read  encrypt data
54     fiobj.read(Buffer, 0, FileSizeInt);
55
56     //Decrypt Data
57     for(int k = 0; k < FileSizeInt; k++)        //$
58     {
59         Buffer[k] = (byte)(Buffer[k] ^ KEY);    //XOR operation
60     }
61
62     // Write decrypted  data
63     foobj.write(Buffer, 0, FileSizeInt);
64
65     foobj.close();
66
67     }// End of while
68
69     fiobj.close();
70     return true;
71 }
72 else
73 {
74     return false;
75 }
76 }
77 }
```